

Lezione 30 Marzo

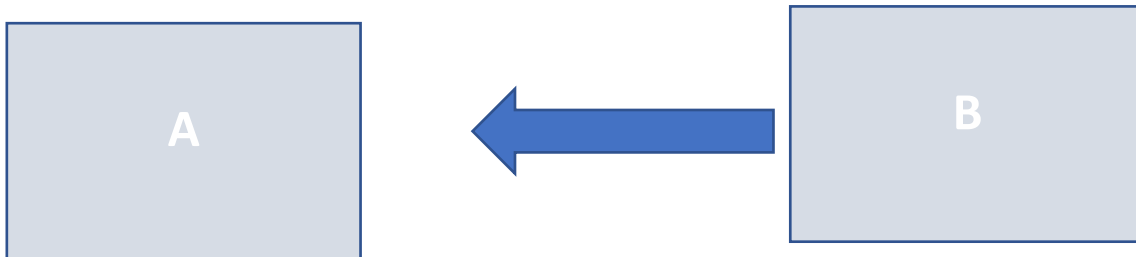
Quando abbiamo una periferica, ogni periferica ha un problema di inizializzazione. Scegliamo un modo di funzionamento che crea un automatismo che fa sì che alcuni segnali vengono gestiti automaticamente. Ad esempio per la PIA se scriviamo/leggiamo i segnali variano in automatico. Per inizializzare ci sono dei bit che messi in un certo registro permettono la configurazione.

Modello di programmazione: come il programmatore vede la periferica, qual è il senso dei registri che la periferica ha.

Abbiamo visto quindi le interruzioni interne e esterne. L'interruzione è un evento che scarica un'azione. L'azione è l'ISR. È semplicemente un evento che fa sì che succede qualcosa. Se questo qualcosa è leggere un carattere allora lo legge e quello dall'altro lato viene sbloccato. Lo sblocco o non sblocco non è legato all'evento ma alla lettura. Se non leggo il sistema rimane bloccato.

Abbiamo inserito un nuovo concetto: risorsa. In ogni risorsa ci vuole un possessore. In taluni casi può esserci bisogno della mutua esclusione, cioè uno solo che possiede la risorsa. Viene gestita da un codice operativo, il TAS. Fa un'azione atomica: legge e se il lock è pari a 0 lo mette ad 1.

Quando arriva l'interruzione, l'altro non si sblocca. Si sblocca solo se leggo. Quindi bisogna distinguere l'interruzione dallo sblocco. Se non leggo il sistema rimane con la mano alzata.



ISRB

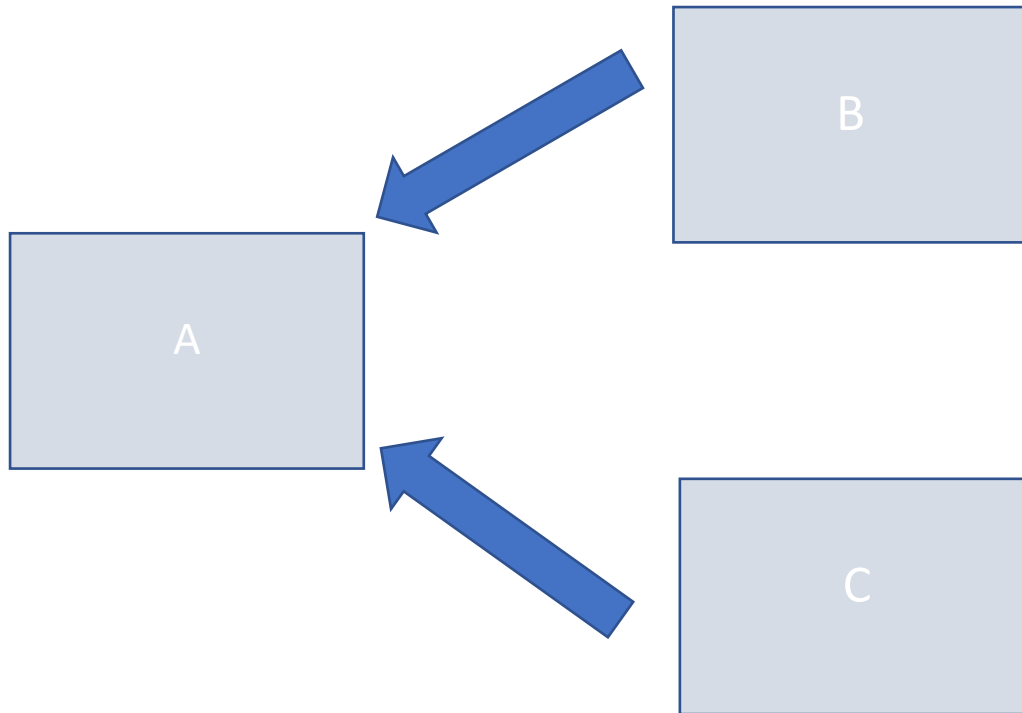
Leggo dato

Incremento contatore

If cont==N

Stop

2)



Avremo sia ISRB e ISRC dentro A. se avessi due buffer non devo gestire la mutua esclusione, non c'è un problema di contesa di risorsa.

3) La cosa più complicata è l'esercizio 3: ho una cosa che lavora in due fasi. La fase 2 la attivo solo se è finita la fase 1. In questo caso devo sapere se la fase 1 è finita.

La fase 1 prevede di prendere tutti gli N caratteri da B e da C. dobbiamo essere sicuri che entrambi abbiamo finito.

Priorità C > priorità B

È difficile quando c'è un evento di contemporaneità. Se finisce prima uno o l'altro non è un problema. Se B sta per vedere C e in quel momento viene interrotto da C, non è un problema vede che B ha finito, ha finito pure lui e finisce la fase 1.

4) vorrei che un messaggio sia letto solo da B o solo da C ma non so chi arriva prima. **Non voglio che sia letto sia da B che da C. il primo che arriva si prende la risorsa e la legge e non mi interessa leggere da C quindi posso farlo non bloccante e buttando i caratteri.** (è una traccia, se invece inizia B ma poi voglio leggere anche C devo farlo bloccante). Mutua esclusione. **C'è una risorsa che devo attribuire a uno dei due. Se assegno la risorsa a B. B legge sempre lui. In buff e' possibile salvare i messaggi di una sola delle due periferiche tra B o C. I messaggi dell'altra periferica, saranno letti e scartati.** La decisione di chi tra B o C debba scrivere in buff viene effettuata in base **all'ordine di ricezione**: se si riceve prima da B, allora B sarà l'unico a dover scrivere in buff. Nota: siamo obbligati a usare TAS come prima istruzione in modo da bloccare chiunque mi prelaiona!!!! Altrimenti se facciamo il doppio controllo sul sender posso essere prelaionato tra il primo if e il TAS, permettendo a chi mi prelaiona di prendersi il sender, e quindi non rispettare la regola che il primo che arriva legge. !!

ISR B:

```

if (TAS(lock) == 0){

    // Aggiorno il possesso solo una volta !

    if(possessiono == None){

        possesso = B

    }

} // Non e' necessario sospendermi se non ottengo il lock, poiché i miei caratteri saranno scartati

var = ReadPIAB //se non sono il sending ogni volta verrò sovrascritto. Leggendo sto mandando gli ACK
a B quindi non rimane sospeso.

// Se il possesso e' mio, vado a scrivere nel buffer !

if(possessiono == B){

    buff[curr_ch] = var

    curr_ch++

    if(curr_ch == N)

        curr_ch = 0

}

```

5) Un sistema è composto da 3 unità, A, B e C, tra loro collegate mediante due periferiche parallele che interconnettono A con B e A con C rispettivamente. A riceve da B e da C messaggi di N caratteri **in maniera alternata**. I messaggi da B e da C devono essere ricevuti **senza interlacciamento dei caratteri**. L'ordine di ricezione inoltre è prefissato dal programmatore. In altri termini è possibile scegliere se ricevere prima un messaggio intero da B e poi un messaggio intero da C o viceversa.

La necessità di non interlacciare i caratteri impone il dover **sospendere una periferica**. L'ordine tuttavia è prefissato dal programmatore: in altri termini, **non è necessario** utilizzare mutua esclusione per impostare la variabile sender/possesso. Al termine della ricezione da una delle due periferiche, la variabile sender/possesso va settata a quella dell'altra periferica, **NON A NONE (alternata)!**

In memoria sender=B già dall'inizio quindi non c'è bisogno della mutua esclusione. Se arriva B vede che è lui e legge. Se nel mentre arriva C vede che sender=B e diventa pending, bloccato in attesa che venga letto.

```

ISR B: if(sender == B){ //Se il sender e' B

    buffB[curr_ch_B] == readPIAB //Leggi il carattere

    curr_ch_B++

    if(curr_ch_B ==N){ // Se la ricezione del messaggio e' terminata, fornisci a C il sender

```

```

curr_ch_B=0 //azzeriamo il contatore per il prox messaggio
sender = C // Assegna sempre sender a C (alternata)!

if(pendingC==1){

    buffC[readPos] = readPIAC
    curr_ch_C++

    pendingC=0 //non è più pendente, ho letto.

}

}

}else{

    pendingB = 1
    RTE

}

```

Se la periferica fosse seriale cambia il modo, ma genera comunque un'interruzione. Varia solo l'inizializzazione.

Lezione 2 30 Marzo

Il processore dentro lo schedino è il processore ARM. Non funziona come il 68k. L'architettura ARM è un'architettura RISC. ARM vende ip-core cioè la specifica.

Soc system on chip è un Sistema messo su un unico circuito integrato. Processore, memoria e periferiche.

Interrupt controller

Il processore è connesso a due bus: APP e HP. Ci sono varie tipologie di periferiche: la seriale e periferiche più lente, ma ci sono periferiche un po' più veloci e che interagiscono con la memoria: DMA, memoria ram e flash e un porto per collegarsi alla memoria esterna.

Tre famiglie di cpu:

- 1) ARM cortex A
- 2) Arm cortex R
- 3) Arm cortex M : microcontrollori

Nei modelli ci sono due set di istruzioni. Ci sono varie motivazioni:

- Retrocompatibilità: devo mantenere le istruzioni vecchie perché se prendo un programma di anni fa deve comunque funzionare
- Thumb

ARM ha 7 modalità operative non solo stato utente e supervisore. Comporta una protezione tra i vari livelli. L'arm può lavorare in diverse modalità operative: Thread mode, Handle mode

Non ci sono registri dati e indirizzi, ma tutti i registri sono general purpos. Derivano da un processore che si chiama MIPS.

Non c'è la parallela, ma possiamo configurare singolarmente i singoli fili e fare una parallela in questo modo.

Multiplexing dei pin: un singolo pin può instradare una periferica, o un'altra, essere un pin di input ecc. può fare tante cose.

STM32CubeIDE è un tool

Traccia

Un sistema è composto da 3 unità, A, B e C, tra loro collegate mediante due periferiche parallele che interconnettono A con B e A con C rispettivamente. Il sistema opera effettuando K iterazioni (con $K > 2$ a scelta dello studente), in ciascuna delle quali A deve ricevere globalmente 2 messaggi di N caratteri da B e 1 messaggio di N caratteri da C (con $N > 2$ a scelta dello studente). I messaggi da B e da C possono essere ricevuti in un ordine qualsiasi ma non deve essere mai possibile ricevere caratteri appartenenti a messaggi diversi intervallati tra di loro.

Un esempio con $K=3$ e $N=3$ di funzionamento del sistema è il seguente:

- Iter 1: msgB_1(1)msgB_1(2)msgB_1(3)msgC_1(1)msgC_1(2)msgC_1(3) msgB_2(1)msgB_2(2)msgB_2(3) (s3)
- Iter 2: msgC_1(1)msgC_1(2)msgC_1(3)msgB_1(1)msgB_1(2)msgB_1(3) msgB_2(1)msgB_2(2)msgB_2(3) (s2)
- Iter 3: :msgC_1(1)msgC_1(2)msgC_1(3)msgB_1(1)msgB_1(2)msgB_1(3) msgB_2(1)msgB_2(2)msgB_2(3) (s2)

ISR_B

```
* Nella sezione critica è solo necessario aggiornare la variabile possesso.
if TAS(lock){

    * Se la variabile possesso non è gestita in lettura da C e
    * se B non ha concluso il numero di messaggi dell' iterazione.

    if (possesso != is_reading_C and end_B != 1 ){

        possesso = is_reading_B

    }

    * Dopo aver aggiornato la variabile di possesso rilascio il lock.
    lock = 0

}

else * Se non ho mai ottenuto il lock me ne esco.
```

RTE

```
* A questo punto del codice ci arrivo solo se ho ottenuto il lock e
* aggiornato/controllato lo stato della variabile di possesso.
* Essendo fuori dalla sezione critica C mi può interrompere in ogni punto
switch(possesso){

    case is_reading_B:

        * Lettura carattere di B ed aggiornamento delle variabili di possesso

    case is_reading_none:

        * Recupera il primo carattere di C se C
        * non è sospeso e non ha terminato i messaggi.

    case is_reading_C:

        * Verifica se C non sia sospeso e nel caso vengono effettuate tutte
        * le operazioni che ISR_C avrebbe eseguito se possesso = is_reading_C

}

RTE
```

- 1) Il caso is reading B è il caso base in cui entriamo se B è riuscito a prendere il canale.
- 2) Case is_reading_None: si verifica se nessuno ha preso possesso e ipotizziamo che B ha terminato di ricevere i 2 messaggi quindi end_B=1. Di conseguenza acquisisce il lock ma non entra nell'if. Mettiamo caso che C essendo più prioritario interrompa B tra queste due istruzioni. Quindi viene lanciata ISRC. ISRC non entra nell'if del TAS perché B è ancora nella sezione critica. Di conseguenza si sospenderà mettendo pending=1 e fa RTE. A questo punto ritorna B e come detto non entra nell'if. A questo punto prenderà la strada del case possesso=is_reading_None e all'interno visto che C è sospeso leggerà il carattere e impostando anche sender=is_reading_C. inoltre è inutile controllare che sia l'ultimo carattere, perché visto che possesso=is_reading_none siamo certi che questo è il primo carattere.
- 3) Caso possesso=is_reading_C: si verifica nel momento in cui A sta leggendo i caratteri di C. arriva una ISRB, prende TAS e viene interrotto. A questo punto viene eseguita ISRC, ma non entra nel TAS e quindi si sospenderà e fa RTE. Ritorna a ISRB, non entra nell'if, rilascia il lock ma entra nel case possesso=is_reading_C. a questo punto vede che C è sospeso e fa la lettura al posto suo. Dentro questa funzione è inutile vedere se endC=1 perché se possesso=is_reading_C significa che sicuramente end_C=0, perché lo stava già servendo. Al massimo con questo carattere che B leggerà per C potrà essere l'ultimo.

C è più prioritario di B quindi significa che se sta eseguendo ISRC, ISRB non potrà mai eseguire perché non può prelaionare C. Quindi B in realtà non potremmo mai entrare nel codice else nel caso in cui TAS fallisce. ISRB può partire solo se ISRC non sta eseguendo. Al contrario invece C ha il problema perché essendo più prioritario può interrompere B e trovare il canale occupato.

```

if(c_suspended == 1 and end_C != 1){
    total_ch_rcvd++
    ctr_char++
    possesso = is_reading_C
    * Prima di sbloccare una periferica conviene
    * aggiornare prima le variabili di stato.
    buff[total_ch_rcvd - 1] = PIABDR
}

```

possesso=is_reading_none.

```

* A questo punto del codice posso leggere serenamente dalla mia PIA ed aggiornare le variabili di stato.
buff[total_ch_rcvd] = PIAADR
total_ch_rcvd++
ctr_char++
if(ctr_char == mex_char){ * Devo controllare se ho completato un intero messaggio
    ctr_char = 0 * Resetta il numero di caratteri del messaggio corrente
    ctr_mex_b++ * Incrementa il numero di messaggi in una iterazione
    possesso = is_reading_none * Resetta il valore della variabile di possesso
    * Se B ha completato il suo numero di messaggi per iterazione
    if(ctr_mex_b == mex_b_per_iter){
        end_B = 1 * Metti end_B = 1 per garantire non legga più nulla.
        * Se anche end_C è 1 allora si è completata una iterazione.
        if(end_C == 1){
            k++
            if(k < n_iter){ * Resetta gli end solo se non ho raggiunto la fine
                end_B = 0
                end_C = 0
                ctr_mex_b = 0
            }
        }
    }
    * Se C non ha terminato il numero di messaggi per iterazioni (oppure end_C non è stato resettato
    * perché sono finite le iterazioni), leggo da C.
    if (end_C != 1 and c_suspended == 1){
        * Le variabili di stato conviene aggiornarle sempre prima di leggere il messaggio (sbloccare la
        * periferica)
        possesso = is_reading_C
        total_ch_rcvd++ * Incrementa il numero totale di byte ricevuti
        ctr_char++ * Incrementa il numero di caratteri ricevuti.
        buff[total_ch_rcvd - 1] = PIABDR
    }
}

```

Possesso=is_reading_B

Se entro qui significa che ho un carattere da leggere da B. quindi lo leggo quindi incremento i contatori

A questo punto vediamo se abbiamo ricevuto tutti i caratteri del singolo messaggio. Se si azzeriamo il contatore e incrementiamo il contatore di messaggi ricevuti in una iterazione. E mettiamo possesso a none

A questo punto controlliamo se abbiamo ricevuto tutti i messaggi in una singola iterazione. Se si mettiamo end_b=1. Se anche end_c era già pari a 1 significa che è terminata un'iterazione e si incrementa il numero di iterazioni.

Ora l'ultima cosa è vedere se abbiamo finito le iterazioni. Se non le abbiamo finito significa che possiamo resettare end_b e end_c e possiamo resettare il contatore dei messaggi ricevuti da B in una singola iterazione.

Ora nell'if per controllare se il messaggio è terminato dobbiamo vedere se C era pendente perché possiamo avere messaggi incrociati, l'importante è che i caratteri di un singolo messaggio siano incrociati.

Se C si era sospeso e non ha finito i messaggi allora il possesso passa a C e leggiamo il suo carattere. La lettura di C va fatta sempre alla fine perché altrimenti potrebbe interrompermi prima di terminare.

C può interrompermi se ancora non si è sospeso? Si certo se lo fa prima che ho messo possesso a none allora si sospenderà e quindi il carattere verrà letto da questa funzione. Se invece lo fa dopo vede che è

non prende il possesso e legge, e poi torneremo a questa funzione per completare. Tutte le variabili sono lette da una sola ISR alla volta quindi saranno aggiornate.

```
if(c_suspended == 1){ * Non è necessario qui controllare se end_C==1
* Qui è necessario effettuare tutte le operazioni che C avrebbe effettuato
* in quanto non si può supporre quanti caratteri C abbia già ricevuto.
corr_char_idx = total_ch_rcvd * Salvo indice del carattere di C
total_ch_rcvd++
ctr_char++
if(ctr_char == mex_char){ * Se C ha completato un messaggio
    ctr_char = 0
    end_C = 1
    possesso = is_reading_none * Resetta la variabile di possesso.
    if(end_B == 1){
        k++
        if(k < n_iter){
            end_B = 0
            end_C = 0
            ctr_mex_b = 0
        }
    }
    if (end_B != 1){
        * In questo caso aggiornamento variabili e lettura potrebbero essere capovolti
        * perché siamo nella ISR_B
        possesso = is_reading_B
        buff[total_ch_rcvd] = PIAADR
        total_ch_rcvd++
        ctr_char++
    }
}
* Andiamo a leggere il messaggio effettivo di C ( total_ch_rcvd era già stato incrementato).
buff[corr_char_idx] = PIABDR
}
```

Case possesso=is_reading_C: come detto prima ISRB potrebbe essere stata prelazionata da ISRC mentre era nella regione critica con possesso=is_reading_C quindi C si è sospeso. Ma potrebbe anche non essere prelazionata. Quindi prima cosa vedo se C è sospeso. Se era sospeso devo fare la lettura come se fossi ISRC.

Ricordiamo che siamo dentro ISRB quindi un carattere da leggere sicuro ce l'ho. Allora se C ha finito devo solo vedere se B non aveva finito, perché se non ha finito sicuro c'è un carattere da leggere da B. la lettura di C facciamo sempre alla fine così non possiamo essere prelazionati. Invece tutti i contatori vengono aggiornati nel flusso normale.

VERSIONE MIA DELLA ISRB: metto subito pending=1 in modo che se vengo prelazionata rimane a 1 e chi mi prelazona sa che deve leggermi. Ovviamente se vengo prelazionata poi non devo leggere quindi devo controllare anche se pending=1 and sending=B prima di leggere.

Se possesso è occupato è inutile fare TAS. Quindi già da prima vedo che se non è none allora vado subito oltre, tanto se non entra nell'if per leggere fa RTE e sarò pendente, altrimenti se era già mio il canale leggerò direttamente.

```
# =====
# ISR_B (Livello Interrupt 3)
# =====
pendingB = 1
```



```
# 1. Tentativo di acquisizione del controllo del canale
```

```
if sender == None and end_B == 0:
```

```
    if TAS(lock) == 0:
```

```
        if sender == None:
```

```
            sender = B
```

```
            lock = 0
```

```
# 2. Controllo e Consumazione
```

```
if sender == B and pendingB == 1:
```

```
    buff[total_ch] = ReadPIAB
```

```
    total_ch = total_ch + 1
```

```
    curr_ch_B = curr_ch_B + 1
```

```
    pendingB = 0
```

```
# 3. Gestione fine messaggio
```

```
if curr_ch_B == N:
```

```
    curr_ch_B = 0
```

```
    ctr_mex_b = ctr_mex_b + 1
```

```
    sender = None          # Il messaggio di B è finito, libero il canale
```

```
# Logica di fine iterazione per B (2 messaggi)
```

```
if ctr_mex_b == 2:
```

```
    end_B = 1
```

```
    if end_C == 1:
```

```
        k = k + 1
```

```
        if k < K:
```

```
            end_B = 0
```

```
            end_C = 0
```

```
            ctr_mex_b = 0
```

```
# 4. Logica di Recupero - SOLO ALLA FINE DEL MESSAGGIO DI B!
```

```
if end_C == 0 and pendingC == 1:
```

```
    sender = C          # Cedo l'intero canale a C
```

```
    pendingC = 0
```

```
total_ch = total_ch + 1  
curr_ch_C = curr_ch_C + 1  
buff[total_ch-1] = ReadPIAC
```

RTE

Non ho bisogno di gestire i casi possesso=`is_reading_C` e `is_reading_None` perché se il canale è di C non faccio proprio TAS quindi se mi interrompe ISRC il canale è libero e legge direttamente lui.

Il caso `is_reading_None` si può verificare se prendo il lock ma non mi prendo il possesso perché ho finito i messaggi. Allora se C mi prelaiona ha un carattere pendente. Ma se io TAS non lo faccio proprio se ho finito i messaggio grazie al primo if, ISRC troverà strada libera per il TAS e quindi leggerà lui.